Aprendizaje automático avanzado
Transformadores

Juan Miguel Santos

Laboratorio de investigación y desarrollo experimental en computación (LIDEC)
Universidad Nacional de Hurlingham

(8 de octubre de 2026)

Transformador

Reuso de matrices Q, K y V (*KV cache*).

- Sea la entrada en tiempo t, que difiere de la anterior en tiempo $t-1$, en 1 *token*, como por ejemplo, cuando la nueva entrada se obtiene desplazando 1 *token* a la izquierda los *tokens* anteriores.
- Dado que la entrada cambió, tendremos que calcular nuevamente las matrices K, Q y V para cada *token*.

Transformador

Reuso de matrices Q, K y V (*KV cache*).

- Pero, si guardamos en memoria los valores de las matrices de los tokens comunes con la entrada anterior $(t-1)$ no tendremos que calcularlos.
- Justamente eso hace **KV-cache** lo cual aumenta notablemente la eficiencia durante inferencia (y según cómo se lleve a cabo, también durante el aprendizaje).

Transformador

Ajuste fino (*fine tunning*).

- Consiste en continuar el aprendizaje de un Transformador ya entrenado. El propósito es continuar el aprendizaje para un dominio específico.
- Por ejemplo, tenemos un Transformador para el español. Esto nos permite, por caso, generar nuevo texto en este idioma.
- Pero, si le agregamos conocimiento específico sobre electrónica (claro está, siempre en español), entonces podremos generar texto que incluya esta área del conocimiento.

Transformador

Ajuste fino (*fine tunning*).

- Así se puede hacer para medicina, abogacía, etc.
- Esto también se puede hacer para un Transformador entrenado con datos multi-lenguajes (durante el entrenamiento recibió entradas en español, inglés, alemán, farsi, etc) y deseamos agregar un lenguaje nuevo.

Transformador

Ajuste fino (*fine tunning*).

Hay cuatro tipos de ajustes finos:

1. Continuar con el entrenamiento adicionando nuevos datos de entrenamiento (por ejemplo, agregando un corpus específico de un área). Se denomina **preentrenamiento continuado** (*continued pretrainig* en inglés).

Transformador

Ajuste fino (*fine tunning*).

Hay cuatro tipos de ajustes finos (continuación):

2. Continuar el entrenamiento pero ajustando solamente un subconjunto de los parámetros y el resto los dejamos tal como fueron calculados hasta el pre-entrenamiento anterior.

- Se denomina **ajuste eficiente de parámetros** (*parameter-efficient finetuning, o PEFT* en inglés).

Transformador

Ajuste fino (*fine tunning*).

Hay cuatro tipos de ajustes finos (continuación):

- Un método PEFT es LoRA (del inglés *Low Rank Adaptation*).
- La idea de LoRA es representar una matriz de pesos W por el producto de dos matrices $W = AB$
  donde
  si $W \in R^{N \times d}$, se elije $A \in R^{N \times r}$ y $B \in R^{r \times d}$ con $r << min(d, N)$.

Transformador

Ajuste fino (*fine tunning*).

Hay cuatro tipos de ajustes finos (continuación):

- Durante el entrenamiento, en vez de reemplazar a W por
  $$W + \Delta W,$$
  lo reemplazamos por
  $$W + AB.$$
- En el paso hacia adelante (durante el entrenamiento), por ejemplo, podemos calcular h según:
  $$h = xW + xAB.$$
- Se aplica a las matrices W^Q, W^K, W^V o W^O.

Transformador

Ajuste fino (*fine tunning*).

Hay cuatro tipos de ajustes finos (continuación):

3. Usar al modelo para construir, por ejemplo, un clasificador.

- Para esto, dejamos fijos los parámetros propios del Transformador y agregamos una capa a la salida del mismo (algo similar a agregar un Cabezal de modelado del lenguaje).
- En este caso, solo se ajustan los parámetros de la capa agregada.

Transformador

Ajuste fino (*fine tunning*).

Hay cuatro tipos de ajustes finos (continuación):

4. Continuar con el entrenamiento del Transformador pero con entradas que incluyen instrucciones.

- Por ejemplo, instrucciones como "A:" refiriéndose a que lo que viene luego de la "A:" es la respuesta a una pregunta introducida en el texto previo a "A:" y que está marcada con "Q:".
- Tanto "A:" como "Q:" y otras tantas instrucciones son representadas como *tokens* particulares.

Transformador

Ajuste fino (*fine tunning*).

Hay cuatro tipos de ajustes finos (continuación):

- Otro ejemplo de instrucción es "TL:NR" (versión compacta de *too long, no read*) usada para la obtención de resúmenes.
- Este tipo de ajuste fino se denomina **ajuste fino supervizado** (o *supervized fine tuning* en inglés) debido que es necesario generar las entradas completas.

Transformador

Ajuste fino (*fine tunning*).

Hay cuatro tipos de ajustes finos (continuación):

- Por ejemplo, para sistemas que responden preguntas, incluir la pregunta precedida por la instrucción "Q:" seguida por la respuesta precedida por la instrucción "A:".
- La construcción de esos conjuntos de entrenamiento es un tema en sí mismo.
- Pueden llevarse a cabo en forma automática (extrayendo preguntas y respuestas, por ejemplo de la web).

Transformador

Ajuste fino (*fine tunning*).

Hay cuatro tipos de ajustes finos (final):

- También se puede llevar a cabo en forma semi-automática:
  - incluyendo preguntas, un contexto extraído de la Wikipedia y dejando a un humano introducir la respuesta, o
  - extrayendo preguntas y posibles respuestas y dejando a un humano seleccionar la más adecuada.

Transformador

El Transformador como una máquina para traducir.

- Un modelo secuencia a secuencia (*seq2seq model o sequence to secuence model* en inglés) es capaz de obtener una secuencia de salida dada una secuencia de entrada, independientemente de la longitud de ambas.
- Estos modelos, para la obtención de la secuencia de salida tienen en cuenta la información contextual de la secuencia de entrada.

Transformador

El Transformador como una máquina para traducir.

- Estos modelos se instancian con redes neuronales artificiales.
- Originalmente, las redes neuronales recurrentes fueron usadas para instanciar este tipo de modelos con una estructura llamada codificador-decodificador.

Transformador

El Transformador como una máquina para traducir.

- Actualmente, los Transformadores son los que instancian este tipo de modelos.
- Constituyen el estado del arte en Traducción Automática (*MT* en inglés).

Transformador

El Transformador como una máquina para traducir.

- Estos sistemas son entrenados a partir de pares de oraciones, una de ellas en el lenguaje de entrada y la otra en el lenguaje de salida.
- Por ejemplo, la primera oración está en español y la segunda oración está en alemán, donde la segunda es la traducción en alemán de la primera en español.

Transformador

El Transformador como una máquina para traducir.

- Los archivos que contienen estos datos de entrenamiento son denominados archivo bi texto.
- Por ejemplo, un archivo bi texto que se usa para entrenar un Transformador como traductor entre kaingang y español tiene el siguiente aspecto:

Ag ri ke han tũg nĩ . Ũn jagtar ag jagãgtãn nĩ . Ũ mỹ tó tũg nĩ , ãjag tỹ ũn jagãgtãn mũ ẽn ti , ãjag mré ke fag mỹ tó tũg nĩ gé . <tab> Tú , por el contrario , cuando des limosna , que tu mano izquierda no sepa lo que hace tu mano derecha ,

Ẽg mỹ ũri ẽg jẽn ti nĩm, ẽg tỹ kurã kar ki ko tĩ ẽn. <tab> danos hoy nuestro pan cotidiano ;

Jykre kórég mág ã tóg nĩ gé, hã ra ã régre jykre pãno sĩ ẽn ne ã mỹ kórég nỹ'. <tab> ¿ Por qué te fijas en la mota del ojo de tu hermano , y no reparas en la viga que hay en el tuyo ? .

. . .

Transformador

El Transformador como una máquina para traducir.

- El traductor está compuesto por dos Transformadores:
  - Un codificador.
  - Un decodificador.

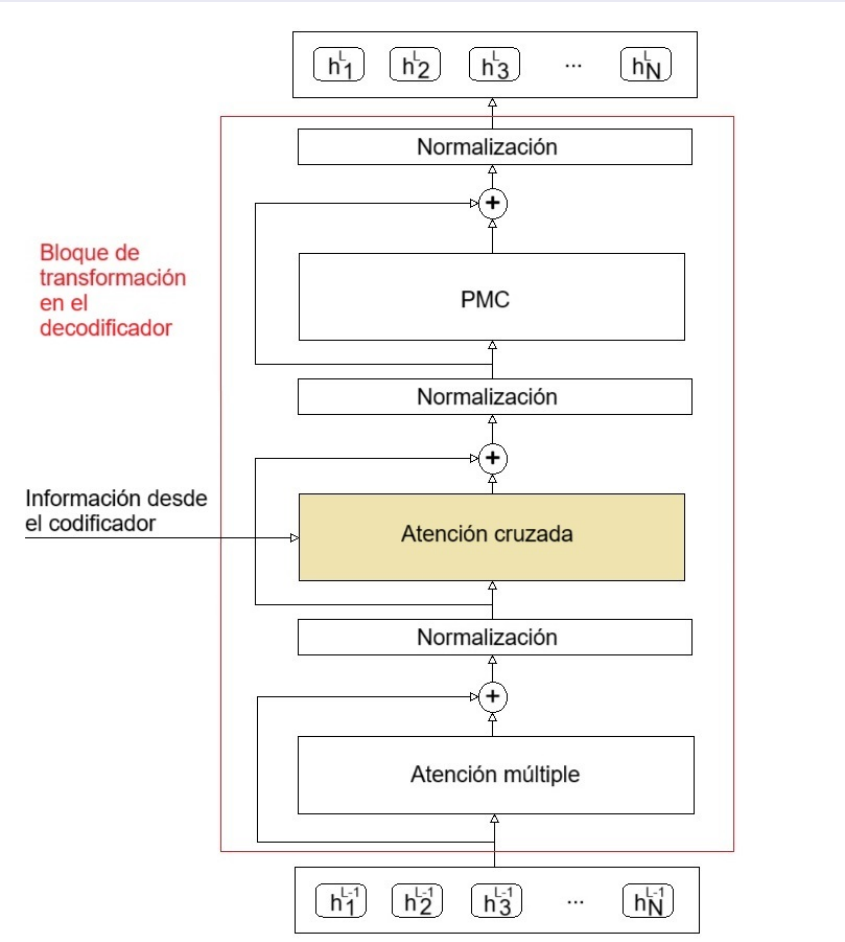
Transformador

El Transformador como una máquina para traducir.

- El codificador tiene la arquitectura del Transformador que ya hemos visto.
- El decodificador tiene una arquitectura similar al del codificador excepto porque cada bloque de transformación tiene incluida una capa más que es denominada **capa de atención cruzada** (o *cross-attention layer* en inglés).

Transformador

El Transformador como una máquina para traducir.



Transformador

El Transformador como una máquina para traducir.

- La capa de atención cruzada tiene una arquitectura similar a la de auto-atención excepto al modo que se obtienen las matrices K y V que es diferente a cómo se obtiene la matriz Q.
- Sea h^{cod} la salida del último bloque del codificador y sea $h^{dec[l-1]}$ la salida del bloque $l-1$ del decodificador.

Transformador

El Transformador como una máquina para traducir.

- La matriz Q de la capa l del decodificador es obtenida según:

$$Q = h^{dec[l-1]} W^Q$$

donde W^Q es la matriz de pesos para consultas del decodificador.

Transformador

El Transformador como una máquina para traducir.

- Las matrices K y V de la capa l del decodificador son obtenidas según:

  $$K = h^{cod} W^K, \text{ y}$$
  $$V = h^{cod} W^V$$

  donde W^K y W^V son las matrices de pesos para claves y valores del decodificador.

Transformador

El Transformador como una máquina para traducir.

- A partir de estas matrices Q, K y V la salida de la capa de atención cruzada es obtenida siguiendo:

$$AtencionCruzada(Q, K, V) = softmax(\frac{QK^t}{\sqrt{d_k}})V.$$

Transformador

El Transformador como una máquina para traducir.

- Un detalle a tener en cuenta es que las capas de auto-atención en el codificador no tienen la matriz QK^t enmascaradas.
- Esto se debe a que la salida de la capas de auto-atención en el decodificador tienen en cuenta todos los *tokens* de la entrada (o sus equivalentes en cada bloque) para construir cada representación contextual a_i.

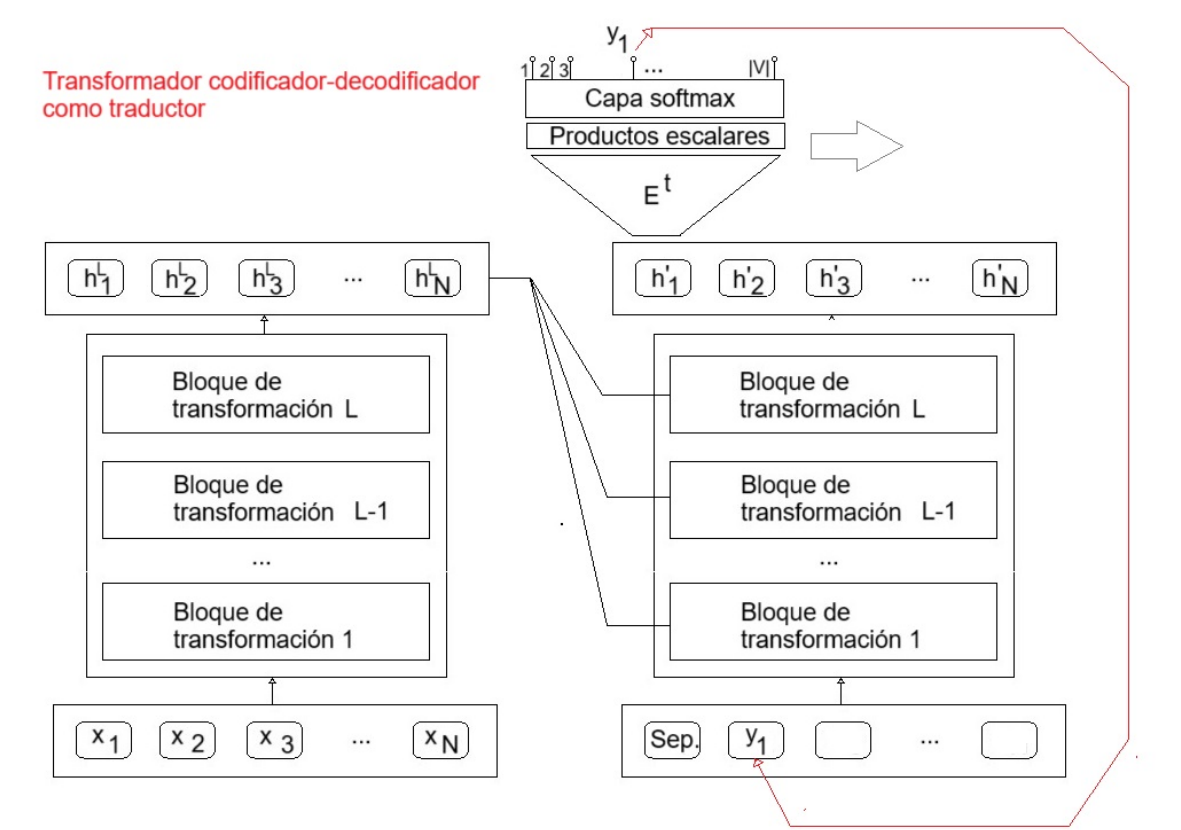
Transformador

El Transformador como una máquina para traducir.

- Mientras que en el codificador la matriz QK^t está enmascarada (la matriz tiene -∞ por arriba de la diagonal) debido a que la atención es causal, en el decodificador la matriz QK^t no está enmascarada ya que la atención es bi direccional.
- Esto es debido a que para obtener la próxima palabra del texto traducido a la salida del decodificador, la capa de auto atención cruzada tiene en cuenta todo el contexto de la entrada sin traducir.

Transformador

El Transformador como una máquina para traducir.



Transformador

El Transformador como una máquina para traducir.

- Durante el entrenamiento se busca maximizar la siguiente probabilidad

$$P(y_1, y_2, ..., y_m / x_1, x_2, ..., x_n)$$

donde $x = (x_1, x_2, ..., x_n)$ es el texto a traducir e $y = (y_1, y_2, ..., y_m)$ es el texto traducido.

Transformador

El Transformador como una máquina para traducir.

- El codificador produce una representación $h_1^L, h_2^L, ..., h_n^L$ a partir de x
  donde L expresa que es último bloque de transformación del codificador.
- El decodificador, en cada tiempo t produce una palabra y_{t+1}:

$$y_{t+1} = decodificador(h^L(t), y_1, y_2, ..., y_t)$$

  $\forall t \in [1 : m]$ y donde $h^L(t)$ expresa la salida en la última capa del codificador a tiempo t.

Transformador

¿De dónde obtener datos para entrenar Transformers?

- El sitio https://commoncrawl.org/ (no comercial) tiene extraído de la web varios billones de páginas.
- La colección de documentos son textos en HTML de las páginas extraídas.

Transformador

¿De dónde obtener datos para entrenar Transformers?

- Hay bases de datos con estos datos depurados como por ejemplo, el trabajo realizado por Raffel et al. en 2020 creando lo que ellos denominaron Colossal CleanCrawled Corpus (C4) a partir de información del Common Crawl del año 2019.
- Puede ser encontrado en https://www.tensorflow.org/datasets/catalog/c4.

Transformador

¿De dónde obtener datos para entrenar Transformers?

- Otros ejemplos de corpus son:
  - The Pile (Gao et al., 2020) y
  - Dolma (Soldaini et al., 2024) que se consideró hasta el 2024 el corpus más grande del inglés con texto de la web, artículos científicos, código de programas, libros, material enciclopédico y datos de medios de comunicación.

Transformador

¿De dónde obtener datos para entrenar Transformers?

- ¿Para qué se usan estos corpus?
- Por ejemplo, los modelos para GPT-3 fueron entrenados con *tokens* obtenidos en la web (429 mil millones de *tokens*), libros (67 mil millones de *tokens*) y Wikipedia (3 mil millones de *tokens*).

Transformador

¿De dónde obtener datos para entrenar Transformers?

- Hay corpus que tienen traducciones en varios idiomas (dos o más).
- Europarl corpus contiene entre 400 mil y 2 millones de oraciones en 23 idiomas que se hablan en Europa.
- United Nations Parallel Corpus que tiene 10 millones de oraciones en los 6 idiomas oficiales de la ONU (inglés, español, chino, francés, ruso y árabe).

Transformador

¿De dónde obtener datos para entrenar Transformers?

- OpenSubtitles corpus hecho a partir de subtítulos para cine y TV.
- ParaCrawl corpus con 223 millones de pares de oraciones entre 23 idiomas e inglés tomados de la web (a través de CommonCrawl).

Transformador

¿De dónde obtener datos para entrenar Transformers?

- Si el corpus de entrenamiento no está dado en pares sino en textos (por ejemplo, a partir de libros completos traducidos) hay que **alinear** las oraciones para producir los pares.
- A veces, dado una traducción de un texto completo no es posible alinear pares de oraciones sino, por ejemplo, que es necesario alinear dos oraciones seguidas con 1, o al revés.
- Esto, frecuentemente se hace mediante algún tipo de algoritmo particular, incluso puede requerir asistencia humana.